

# **ECE/CS 552 Fall 2024: Final Project Report**

## **WISC\_F24**

**Dhruv Bittal Singhi**  
**Shraddha Singh**  
**Sumukha Madodi**

# Design and Implementation of a WISC-F24 Processor

# Introduction

This project involved the systematic design and implementation of a processor based on the WISC-F24 Instruction Set Architecture (ISA). Executed in three progressive phases, the project aimed to incrementally enhance the processor's functionality and complexity while adhering to the ISA's specifications. The design culminated in a 5-stage pipelined processor featuring control blocks for hazard detection, data forwarding, and branch prediction, alongside the integration of L1 caches for both instruction and data storage to optimize performance and efficiency.

## Phase 1: Single-Cycle Processor

The first phase focused on designing a single-cycle processor based on the WISC-F24 ISA, which features a 16-bit data path, 16 registers with a 3-bit FLAG register, and a load/store architecture. Operations are categorized into Compute, Memory, and Control classes. The design was implemented in Verilog and verified using ModelSim and Icarus simulators, forming the foundation for subsequent phases.

## Phase 2: 5-Stage Pipelined Processor

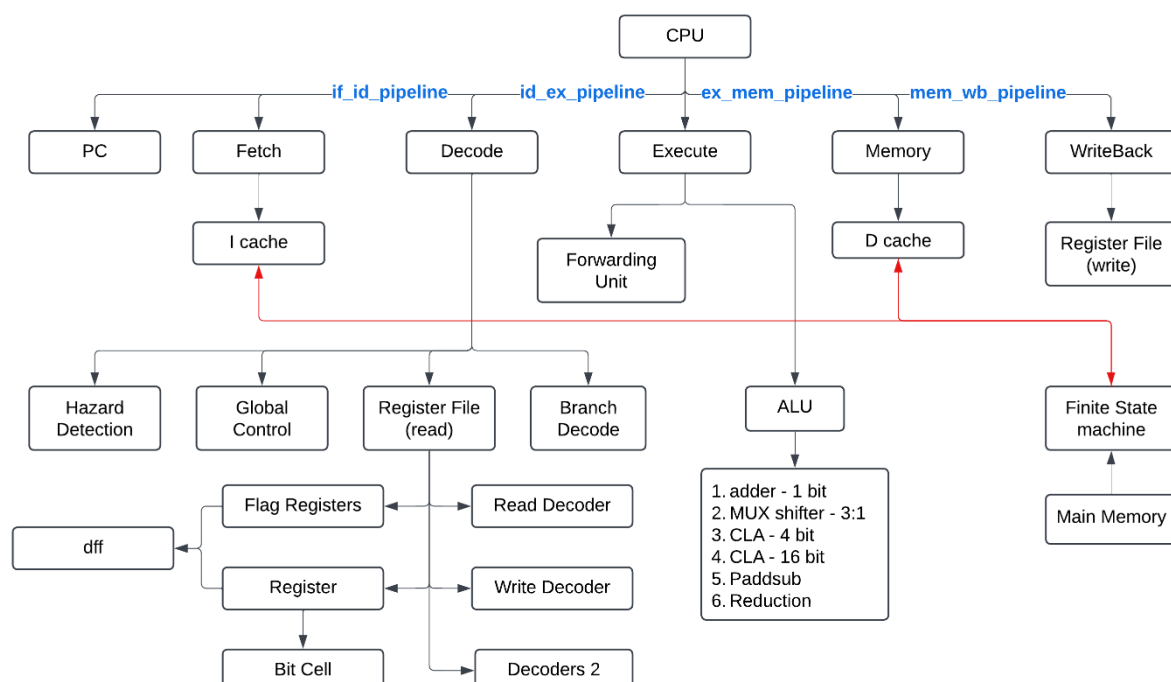
Building upon the single-cycle design, the second phase focused on developing a 5-stage pipelined processor. Functional blocks from Phase 1 were reused to maintain modularity. The primary challenges included implementing pipeline control, which encompassed:

- Detection and mitigation of control and data hazards (stalls, flushes, and forwarding).
- Modifications to the control and data paths for efficient data forwarding, including register file and flag bypassing.

### Phase 3: Cache Hierarchy Integration

In the final phase, we enhanced the processor by integrating a cache hierarchy. This involved the design and implementation of instruction (I-cache) and data (D-cache) modules, along with a cache miss controller to manage seamless communication between the caches, the processor pipeline, and main memory.

## Top – level block diagram



## Responsibility and Task Allocation

### Phase 1

We developed a single-cycle processor in Verilog, adhering strictly to the given Verilog coding standards. Key modules implemented include a 1-bit adder, 3-to-1 MUX shifter, bit cell, 4-bit and 16-bit carry-lookahead adders (CLA), ALU modules, main CPU module, global control unit, paddsub module, read and write decoders (4-to-16), reduction module, registers, and register file.

Each module was rigorously tested using comprehensive testbenches to verify all possible cases. Once validated, we integrated these modules into complete CPU and ALU modules, forming our single-cycle processor. The integrated system was successfully tested against provided test files to ensure correctness.

### Phase 2

We transformed our single-cycle processor into a 5-stage pipelined processor by adding pipeline stages and corresponding registers. The design included a forwarding unit to minimize the number of stalls, and a hazard detection unit to detect and put the required stalls in the pipeline. These enhancements were integrated into the main CPU.v file.

The pipelined processor was tested using the provided test cases to ensure accurate functionality, with thorough debugging carried out after integration.

### Phase 3

We enhanced the 5-stage pipelined processor by integrating instruction (I-cache) and data (D-cache) caches to improve performance. Finite state machines (FSMs) were employed to handle cache misses efficiently. The design included a cache controller, an interface for integrating the caches with the pipelined processor, memory modules, and a testbench for comprehensive validation.

The caches were integrated into the CPU.v file, and the entire system was rigorously tested using the provided testbench, with debugging performed collaboratively to ensure proper functionality.

### Team Contributions with Percentage Distribution

| Phase                           | Team Member | Tasks                                                                                   | Percentage Contribution   |
|---------------------------------|-------------|-----------------------------------------------------------------------------------------|---------------------------|
| Phase 1: Single-Cycle Processor | Sumukha     | 1-bit adder, bit cell, CLA-4bit, CLA-16bit, reduction module, respective testbenches.   | 33.6%                     |
|                                 | Shraddha    | paddsub, read decoder, write decoder, registers, register file, respective testbenches. | 32.7%                     |
|                                 | Dhruv       | ALU, global control, 3-to-1 MUX shifter, respective testbenches.                        | 33.7%                     |
|                                 | All         | Integration, CPU, testing, and debugging.                                               | Shared Effort (Not Split) |
| Phase 2: Pipelined Processor    | Sumukha     | EX_MEM_pipeline_reg, ID_EX_pipeline_reg, IF_ID_pipeline_reg, MEM_WB_pipeline_reg.       | 33.5%                     |
|                                 | Shraddha    | Forwarding unit.                                                                        | 33.8%                     |
|                                 | Dhruv       | Hazard detection unit.                                                                  | 32.7%                     |
|                                 | All         | Integration, CPU, testing, and debugging.                                               | Shared Effort (Not Split) |
| Phase 3: Processor with Cache   | Sumukha     | Caches                                                                                  | 32.9%                     |
|                                 | Shraddha    | Memory hierarchy                                                                        | 33.6%                     |
|                                 | Dhruv       | Cache fill FSM.                                                                         | 33.5%                     |
|                                 | All         | Integration of caches into CPU, testing, and debugging.                                 | Shared Effort (Not Split) |

All team members collaborated to integrate individual modules and components into the CPU.v file. We debugged the CPU design and ensured it met testing requirements. Additionally, we integrated the caches into the CPU.v file and finalized the system after debugging.

## Special Features, Optimization Efforts, and Challenges

**Special Features:** In Phase 2, additional test cases were written to verify the functionality of the BR waiting for the register, as the original testcase did not cover this condition. Additionally, the testcases also did not test load-to-use stalls, so we added a testcase for that as well. The provided test files generalized branch stalls for any flag writes, so we refined them to check only the relevant flags, optimizing performance by avoiding unnecessary stalls.

**Optimization Efforts:** Efforts focused on refining branch stall handling by ensuring only necessary flags triggered stalls. This targeted approach improved efficiency compared to the generalized stalling in the original design.

**Challenges Faced:**

- While implementing the Phase 3 cache module, we realized that updating the Least Recently Used (LRU) on a hit or write required knowledge of the previous LRU bit. Initially, the MetadataWay arrays were designed to output 'z' during writes. After discussing the issue with our professor, we received updated files for the MetadataWay array to resolve this problem.
- Integrating individual modules was challenging due to limited experience and differing coding styles among team members. Aligning components to function together took significant effort.
- Debugging was hindered by the large number of signals and ports, making waveform analysis difficult.
- Compilation became problematic with large files and complex code. Tools like ICARUS Verilog timed out, and Modelsim Lite crashed frequently, forcing us to work in the lab despite harsh weather. Additionally, strict Verilog rules conflicted with our ongoing learning of SystemVerilog.

## Completeness

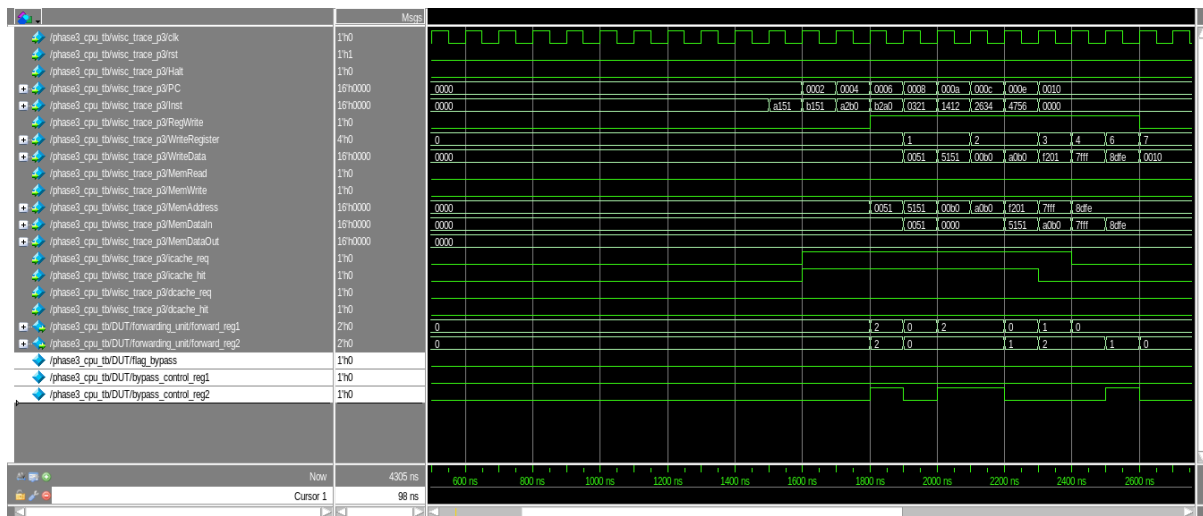
In Phase 1, we overlooked hardwiring register zero to always be zero. In Phase 2, we successfully met all the specified requirements. For Phase 3, we have reviewed all the requirements and believe they have been fully addressed.

## Testing

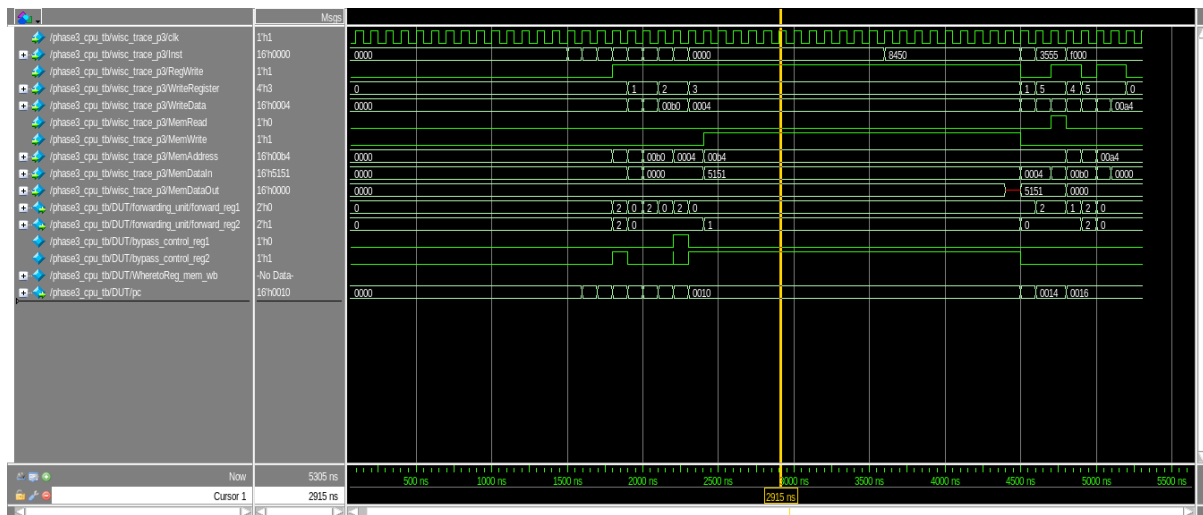
We tested the correctness of our design using a combination of block-level and full-chip methodologies. All modules related to compute-type instructions and the register file were verified using self-checking testbenches to ensure accurate functionality. We tested for corner cases such as Addition and subtraction saturation on overflow. The memory hierarchy was thoroughly tested by simulating individual cases and observing the data outputs on the transcript. This included scenarios such as individual cache misses, pipelined misses, and store operations.

# Simulation Waveforms: Phase 3

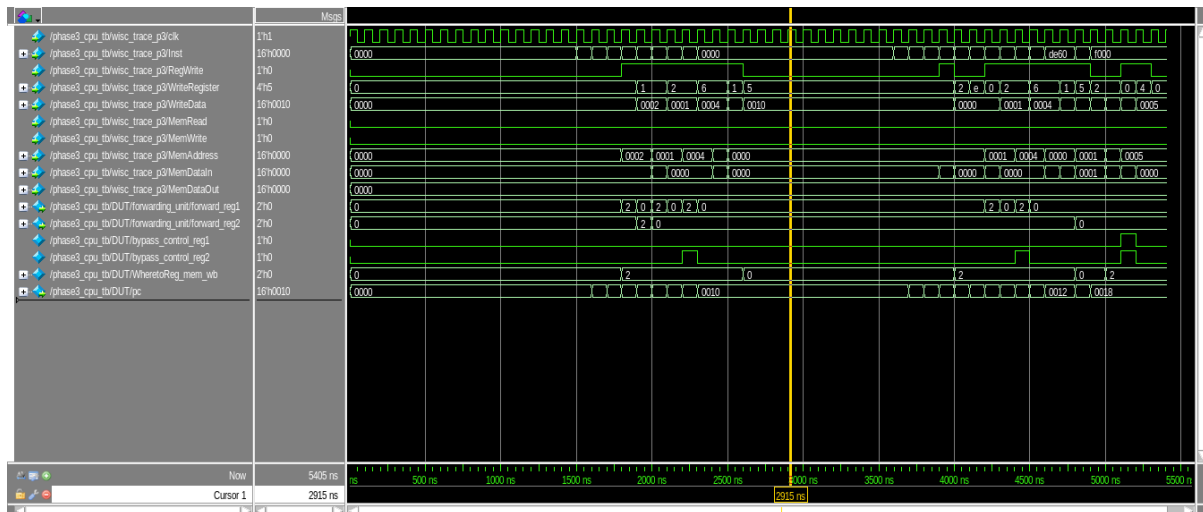
## Test 1



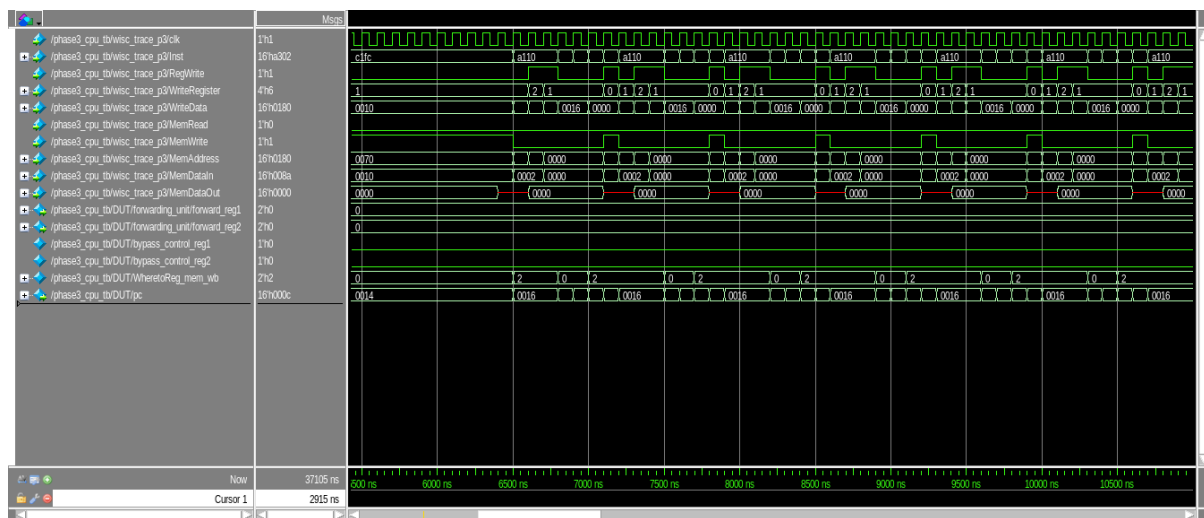
## Test 2



## Test 3



## Test 4



## Simulation Results

### Phase 1 Results

| Program   | Sim Cycles | Instruction Count | Outcome |
|-----------|------------|-------------------|---------|
| Program 1 | 13         | 11                | Correct |
| Program 2 | 14         | 12                | Correct |
| Program 3 | 21         | 19                | Correct |

### Phase 2 Results

| Program   | Sim Cycles | Instruction Count | Outcome |
|-----------|------------|-------------------|---------|
| Program 1 | 16         | 11                | Correct |
| Program 2 | 18         | 11                | Correct |
| Program 3 | 28         | 18                | Correct |

### Phase 3 Results

| Program   | Sim Cycles | Instruction Count | Outcome |
|-----------|------------|-------------------|---------|
| Program 1 | 43         | 11                | Correct |
| Program 2 | 53         | 31                | Correct |
| Program 3 | 54         | 18                | Correct |
| Program 4 | 371        | 164               | Correct |